Q1. Decisions with Many Boundaries (6 Marks)

1. Prediction before execution

Subsidy rules:

- 100 units or less – Full subsidy.
- 101–200 units – Partial subsidy only with a ration card.
- Above 200 units – No subsidy unless a family member is above 80 years.

Household	Units	Ration Card	Member above 80	Expected Result
H1	80	Yes	No	Full subsidy
H2	100	No	No	Full subsidy
H3	101	Yes	No	Partial subsidy
H4	150	No	No	No subsidy
H5	200	Yes	No	Partial subsidy
H6	201	Yes	No	No subsidy
H7	250	No	Yes	Special subsidy
H8	300	Yes	No	No subsidy

2. Python Program – Separate if Statements

```
def subsidy_separate(units, ration, elderly):
    results = []

    if units <= 100:
        results.append("Full subsidy")

    if 101 <= units <= 200 and ration:
```

```
        results.append("Partial subsidy")

    if units > 200 and elderly:
        results.append("Special subsidy")

    if len(results) == 0:
        results.append("No subsidy")

    return results



print(subsidy_separate(100, False, False))
print(subsidy_separate(150, True, False))
print(subsidy_separate(250, False, True))
```

Output:

```
['Full subsidy']
['Partial subsidy']
['Special subsidy']
```

3. Python Program – if-elif-else

```
def subsidy(units, ration, elderly):

    if units <= 100:
        return "Full subsidy"
```

```
    elif 101 <= units <= 200 and ration:
        return "Partial subsidy"

    elif units > 200 and elderly:
        return "Special subsidy"

    else:
        return "No subsidy"



households = [
    (80, True, False),
    (100, False, False),
    (101, True, False),
    (150, False, False),
    (200, True, False),
    (201, True, False),
    (250, False, True),
    (300, True, False)
]

for h in households:
    print(h, ":", subsidy(*h))
```

Output:

(80, True, False) : Full subsidy

(100, False, False) : Full subsidy

(101, True, False) : Partial subsidy

(150, False, False) : No subsidy

(200, True, False) : Partial subsidy

(201, True, False) : No subsidy

(250, False, True) : Special subsidy

(300, True, False) : No subsidy

```
            return "Special subsidy"
        else:
            return "No subsidy"

households = [
    (80, True, False),
    (100, False, False),
    (101, True, False),
    (150, False, False),
    (200, True, False),
    (201, True, False),
    (250, False, True),
    (300, True, False)
]

for h in households:
    print(h, ":", subsidy(*h))
```

```
['Full subsidy']
['Partial subsidy']
['Special subsidy']
(80, True, False) : Full subsidy
(100, False, False) : Full subsidy
(101, True, False) : Partial subsidy
(150, False, False) : No subsidy
(200, True, False) : Partial subsidy
(201, True, False) : No subsidy
(250, False, True) : Special subsidy
(300, True, False) : No subsidy
```

4. Analysis

Separate if statements execute independently. Therefore, more than one condition can be true, resulting in multiple decisions.

The if-elif-else structure checks conditions in sequence and executes only the first matching block.

5. Design Decision

I prefer if-elif-else because it makes the subsidy conditions easier to understand, maintain and modify.

6. Alternative Concept – Match Case

```
def subsidy_match(units, ration, elderly):

    if units <= 100:
        category = 1
    elif units <= 200 and ration:
        category = 2
    elif units > 200 and elderly:
        category = 3
    else:
        category = 4

    match category:
        case 1:
            return "Full subsidy"
        case 2:
            return "Partial subsidy"
        case 3:
            return "Special subsidy"
        case _:
            return "No subsidy"

print(subsidy_match(100, False, False))
print(subsidy_match(150, True, False))
```

Complexity Analysis

Approach	Time Complexity	Space Complexity
if-elif-else	O(1)	O(1)
Match-case	O(1)	O(1)
Lookup list with k slabs	O(k)	O(k)

Q2. Loops that Count Attempts (4 Marks)

1. Problem Explanation

A bank allows three OTP attempts.

- Correct OTP – Login successful.
- Wrong OTP – Attempt count increases.
- Blank OTP – Does not count.
- Three wrong attempts – Account locked for 30 minutes.

2. Using While Loop

```
correct_otp = "1234"
attempts = 0

while attempts < 3:

    otp = input("Enter OTP: ")

    if otp == "":
        print("Blank entry. Try again.")
```

```
        continue

    attempts += 1

    if otp == correct_otp:
        print("Login successful")
        break

    else:
        print("Wrong OTP")

else:
    print("Account locked for 30 minutes")
```

3. Using For Loop

```
correct_otp = "1234"

for i in range(3):

    otp = input("Enter OTP: ")

    if otp == "":
        print("Blank entry. Try again.")
        continue
```

```
    if otp == correct_otp:
        print("Login successful")
        break

    else:
        print("Wrong OTP")

else:
    print("Account locked for 30 minutes")
```

Note: In the for-loop version, blank entries consume an iteration of range(3). Therefore, it does not fully satisfy the requirement that blank entries should not count. A corrected version is:

```
correct_otp = "1234"
attempts = 0

for i in range(3):

    while True:
        otp = input("Enter OTP: ")

        if otp != "":
            break

        print("Blank entry. Try again.")
```

```
    attempts += 1

    if otp == correct_otp:
        print("Login successful")
        break

    print("Wrong OTP")

else:
    print("Account locked for 30 minutes")
```

4. Explanation of break, continue and pass

Statement	Purpose
break	Terminates the loop
continue	Skips current iteration
pass	Does nothing; acts as a placeholder

Example of pass:

```
if attempts == 2:
    pass
```

5. While vs For

A while loop is more natural because the number of attempts depends on valid OTP entries rather than the number of input operations.

If the counter is updated before checking for blank input, blank entries will incorrectly count as attempts.

6. Changed Condition

If the customer answers the security question correctly, two additional attempts are allowed.

```
correct_otp = "1234"
attempts = 0

security = input("Security question correct? (yes/no): ")

max_attempts = 5 if security.lower() == "yes" else 3

while attempts < max_attempts:

    otp = input("Enter OTP: ")

    if otp == "":
        print("Blank entry")
        continue

    attempts += 1

    if otp == correct_otp:
        print("Login successful")
        break

    print("Wrong OTP")
```

```
else:
    print("Account locked for 30 minutes")
```

7. Alternative Concept – Recursion

```
def verify(attempts, maximum=3):

    if attempts == maximum:
        print("Account locked")
        return

    otp = input("Enter OTP: ")

    if otp == "":
        return verify(attempts, maximum)

    if otp == "1234":
        print("Login successful")
        return

    print("Wrong OTP")
    verify(attempts + 1, maximum)
verify(0)
```

Complexity Analysis

Method	Time	Space
While	O(a)	O(1)
For	O(a)	O(1)
Recursion	O(a)	O(a)

Here, a represents the maximum number of attempts.

Q3. Vehicle Registration Number Validator (6 Marks)

1. Problem Explanation

A valid vehicle number contains:

- Two state letters.
- Two district digits.
- One or two series letters.
- Four final digits.

Example: TN09AB1234.

2. Complete Python Program

```
def normalise(text):
    text = text.replace(" ", "")
    text = text.replace("-", "")
    return text.upper()



def is_valid(reg):
```

```
    if len(reg) not in (9, 10):
        return False

    if not reg[0:2].isalpha():
        return False

    if not reg[2:4].isdigit():
        return False

    if len(reg) == 9:
        if not reg[4:5].isalpha():
            return False
    else:
        if not reg[4:6].isalpha():
            return False

    if not reg[-4:].isdigit():
        return False

    return True


def format_number(reg):
```

```
    if not is_valid(reg):
        return "INVALID"

    return reg[:2] + " " + reg[2:4] + " " + reg[4:-4] + " " + reg[-4:]


numbers = [
    "tn 09 ab 1234",
    "KA-01-M-0007",
    "TN9AB1234",
    "TN 22 1234 AB",
    "KL 07 C 5678",
    "AP 39 BC 9876"
]

for raw in numbers:

    reg = normalise(raw)

    print(raw, "->", reg,
          format_number(reg))
```

3. Expected Output

tn 09 ab 1234 -> TN09AB1234 TN 09 AB 1234

KA-01-M-0007 -> KA01M0007 KA 01 M 0007

TN9AB1234 -> TN9AB1234 INVALID

TN 22 1234 AB -> TN221234AB INVALID

KL 07 C 5678 -> KL07C5678 KL 07 C 5678

AP 39 BC 9876 -> AP39BC9876 AP 39 BC 9876

```
    if not is_valid(reg):
        return "INVALID"

    return reg[:2] + " " + reg[2:4] + " " + reg[4:-4] + " " + reg[-4:]


numbers = [
    "tn 09 ab 1234",
    "KA-01-M-0007",
    "TN9AB1234",
    "TN 22 1234 AB",
    "KL 07 C 5678",
    "AP 39 BC 9876"
]

for raw in numbers:

    reg = normalise(raw)

    print(raw, "->", reg,
          format_number(reg))
```

```
tn 09 ab 1234 -> TN09AB1234 TN 09 AB 1234
KA-01-M-0007 -> KA01M0007 KA 01 M 0007
TN9AB1234 -> TN9AB1234 INVALID
TN 22 1234 AB -> TN221234AB INVALID
KL 07 C 5678 -> KL07C5678 KL 07 C 5678
AP 39 BC 9876 -> AP39BC9876 AP 39 BC 9876
```

4. Explanation of String Immutability

Strings in Python are immutable. This means their original characters cannot be modified after creation.

Example:

text = "hello"

text.upper()

print(text)

Output:

hello

The original string remains unchanged because upper() returns a new string.

Correct method:

```
text = text.upper()

print(text)
```

Output:

HELLO

5. String Slicing

For a 9-character string:

```
s = "TN09M1234"

print(s[-4:])

print(s[4:-4])
```

Output:

1234

M

For a 10-character string:

```
s = "TN09AB1234"

print(s[-4:])

print(s[4:-4])
```

Output:

1234

AB

6. Alternative Concept – Regular Expression

```
import re


def validate(reg):
   return bool(
      re.fullmatch(
         r"[A-Z]{2}[0-9]{2}[A-Z]{1,2}[0-9]{4}",
         reg
      )
   )
print(validate("TN09AB1234"))
print(validate("TN9AB1234"))
```

Output:

True

False



Complexity Analysis

Method	Time Complexity	Space Complexity
String slicing	O(L)	O(1) auxiliary space
Regular expression	O(L)	O(L) worst-case auxiliary space

Here, L represents the length of the registration number.

Q4. Building Long Text: + or join()? (6 Marks)

1. Problem Explanation

A mobile operator generates monthly SMS bills containing thousands of transaction lines.

Repeated string concatenation can become inefficient because Python strings are immutable.

2. Complete Python Program

```
import time


items = [f"Item{i}:Rs{i % 50}" for i in range(200_000)]


def build_plus(parts):


    text = ""


    for p in parts:

        text += p + ";"


    return text

def build_join(parts):
```

```
    return ";".join(parts) + ";"


start = time.perf_counter()
a = build_plus(items)
t_plus = time.perf_counter() - start

start = time.perf_counter()
b = build_join(items)
t_join = time.perf_counter() - start

print("same text:", a == b, "| length:", len(b))

print(f"+= time: {t_plus:.4f} s")
print(f"join time: {t_join:.4f} s")
```

3. Expected Output

same text: True | length: 3048890

+= time: (varies)

join time: (varies)

4. Explanation

The + operator creates new strings during concatenation because strings are immutable.

The join() method combines multiple strings efficiently in one operation.

Therefore, join() is generally recommended when combining a large number of strings.

5. Three Implementations

Method 1 – Repeated +=

```
def build_plus(parts):
    text = ""

    for p in parts:
        text += p + ";"

    return text
```

Method 2 – join()

```
def build_join(parts):
    return ";".join(parts) + ";"
```

Method 3 – f-string

```
def build_fstring(parts):

    text = ""

    for p in parts:
        text += f"{p};"

    return text
```

6. Performance Measurement

Run the following code for different input sizes:

```
import time


for n in [1000, 10000, 200000]:


    items = [f"Item{i}" for i in range(n)]


    start = time.perf_counter()

    build_plus(items)

    plus_time = time.perf_counter() - start


    start = time.perf_counter()

    build_join(items)

    join_time = time.perf_counter() - start


    print("Items:", n)

    print("Plus:", plus_time)

    print("Join:", join_time)
```

For the report, run this experiment three times and calculate the average.

Average time:

Average = (Time 1 + Time 2 + Time 3) / 3

7. Alternative Concept – StringIO

```
from io import StringIO
```

```
def build_stringio(parts):

    output = StringIO()

    for p in parts:
        output.write(p)
        output.write(";")

    return output.getvalue()

items = ["Tea", "Coffee", "Juice"]

print(build_stringio(items))
```

Output:

Tea;Coffee;Juice;

Complexity Analysis

Method	**Time Complexity**	**Space Complexity**
Repeated +=	Up to $O(n^2)$	O(n)
join()	O(n)	O(n)
StringIO	O(n)	O(n)
Direct file writing	O(n)	O(1) auxiliary buffer

Here, n represents the total number of characters or the total size of the generated statement.

Q5. Functions, Parameters and Scope (7 Marks)

1. Problem Explanation

The function below contains a mutable default parameter.

```
def add_item_buggy(item, order=[]):
   order.append(item)
      return order
```

The default list is created only once when the function is defined.

Therefore, all function calls without an explicit list share the same list.

2. Complete Code – TODO 1

```
def add_item_buggy(item, order=[]):

    order.append(item)

    return order



def add_item(item, order=None):

    if order is None:
        order = []

    order.append(item)

    return order
print(add_item_buggy("Tea"), add_item_buggy("Vada"))
print(add_item("Tea"))
```

print(add_item("Vada"))

3. Expected Output

['Tea', 'Vada'] ['Tea', 'Vada']

['Tea']

['Vada']

```
def add_item_buggy(item, order=[]):

    order.append(item)

    return order


def add_item(item, order=None):

    if order is None:
        order = []

    order.append(item)

    return order
print(add_item_buggy("Tea"), add_item_buggy("Vada"))
print(add_item("Tea"))
print(add_item("Vada"))
```

```
['Tea', 'Vada'] ['Tea', 'Vada']
['Tea']
['Vada']
```

4. Explanation of Default Parameters

The buggy function uses the same list for every call.

The corrected function uses None as its default value and creates a new list whenever the function is called without an order.

5. Positional, Keyword and Default Parameters

Positional parameters:

```
def student(name, age):
    print(name, age)
```

```
student("Arun", 20)
```

Keyword parameters:

```
student(age=20, name="Arun")
```

Default parameters:

```
def greet(name="Student"):
    print("Hello", name)


greet()
```

6. Returning Multiple Values

```
def calculate(amount):

    gst = amount * 0.18

    total = amount + gst

    return total, gst

total, gst = calculate(1000)
print("Total:", total)
print("GST:", gst)
```

Output:

Total: 1180.0

GST: 180.0

7. Local, Global and Nonlocal Variables

Local variable:

```
def display():
    x = 10
    print(x)

display()
```

A local variable is declared inside a function and can be accessed within that function.

Global variable:

```
x = 10

def display():
   global x
    x = 20
display()
print(x)
```

Output:

20

A global variable is declared outside a function. The global keyword allows a function to modify it.

Nonlocal variable:

```
def outer():
```

```
    x = 10

    def inner():

        nonlocal x
        x = 20

    inner()

    print(x)


outer()
```

Output:

20

The nonlocal keyword allows an inner function to modify a variable belonging to its enclosing function.

8. Why Global Variables Are Usually Avoided

Global variables can be modified by different functions, making programs difficult to understand, test and debug.

Local variables and function parameters provide better control and maintainability.

9. Testing with Three Customers

```
customer1 = add_item("Tea")

customer2 = add_item("Coffee")
```

customer3 = add_item("Vada")

print("Customer 1:", customer1)

print("Customer 2:", customer2)

print("Customer 3:", customer3)

Output:

Customer 1: ['Tea']

Customer 2: ['Coffee']

Customer 3: ['Vada']

```
def add_item(item, order=None):
    if order is None:
        order = []
    order.append(item)
    return order

customer1 = add_item("Tea")
customer2 = add_item("Coffee")
customer3 = add_item("Vada")

print("Customer 1:", customer1)
print("Customer 2:", customer2)
print("Customer 3:", customer3)
```

```
Customer 1: ['Tea']
Customer 2: ['Coffee']
Customer 3: ['Vada']
```

10. Alternative Concept – Explicit List

def add_item(item, order):

order.append(item)

```
    return order


order1 = []
order2 = []


add_item("Tea", order1)
add_item("Coffee", order2)


print(order1)
print(order2)
```

Output:

['Tea']

['Coffee']

```
def add_item(item, order):

        order.append(item)

        return order

order1 = []
order2 = []

add_item("Tea", order1)
add_item("Coffee", order2)

print(order1)
print(order2)


['Tea']
['Coffee']
```

Complexity Analysis

Method	Time Complexity	Space Complexity
Default None	O(1) per item	O(n)
Explicit list	O(1) per item	O(n)
Closure	O(1) per item	O(n)

Q6. Recursion and Its Limits (8 Marks)

1. Problem Explanation

A devotee can climb either one or two steps at a time.

We need to calculate the number of different ways to reach the top.

The formula is:

ways(n) = ways(n-1) + ways(n-2)

2. Base Case

```
if n <= 1:
    return 1
```

Base cases stop recursion when the staircase has zero or one step.

3. Recursive Case

```
return ways(n-1) + ways(n-2)
```

The function calculates the number of ways by considering one-step and two-step movements.

4. Complete Python Program – Naive and Memoisation

```
from functools import lru_cache
calls = 0
def ways_naive(n):

    global calls
    calls += 1

    if n <= 1:
```

```
        return 1
return ways_naive(n-1) + ways_naive(n-2)
@lru_cache(maxsize=None)
def ways_memo(n):

    global calls
    calls += 1

    if n <= 1:
        return 1

    return ways_memo(n-1) + ways_memo(n-2)



calls = 0

print("naive:", ways_naive(25), "calls =", calls)

calls = 0

print("memo :", ways_memo(25), "calls =", calls)
```

5. Expected Output

naive: 121393 calls = 242785

memo : 121393 calls = 26

```
    global calls
    calls += 1

    if n <= 1:
        return 1
    return ways_naive(n-1) + ways_naive(n-2)

@lru_cache(maxsize=None)
def ways_memo(n):

    global calls
    calls += 1

    if n <= 1:
        return 1

    return ways_memo(n-1) + ways_memo(n-2)


calls = 0

print("naive:", ways_naive(25), "calls =", calls)

calls = 0

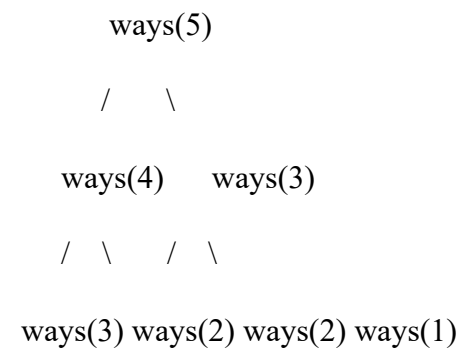
print("memo :", ways_memo(25), "calls =", calls)
```

```
naive: 121393 calls = 242785
memo : 121393 calls = 26
```

6. Answers for n = 1 to 6

Number of Steps	Number of Ways
1	1
2	2
3	3
4	5
5	8
6	13

7. Call Tree for n = 5



The same subproblems are calculated repeatedly.

For example, ways(3) is calculated more than once.

Memoisation stores previously calculated results and avoids repeating calculations.

8. Function Call Comparison

n	Naive Calls	Memoised Calls
10	177	11
20	21,891	21
25	242,785	26
30	2,692,537	31

9. Recursion Limit

Python has a recursion limit to prevent excessive function calls from exhausting the call stack.

```
import sys


print("Recursion limit:", sys.getrecursionlimit())


print(ways_memo(1000))
```

This can produce a RecursionError because the recursive calls exceed Python's recursion depth limit.

Even though memoisation reduces repeated calculations, it does not eliminate recursion depth.

10. Alternative Concept – Iterative Method

```
def ways_loop(n):

    if n <= 1:
        return 1

    a = 1
    b = 1

    for i in range(2, n + 1):

        c = a + b

        a = b
        b = c

    return b



print(ways_loop(1000))
```

The iterative method calculates the result without recursive function calls.

Complexity Analysis

Method	**Time Complexity**	**Space Complexity**
Naive recursion	$O(\varphi^n)$	O(n)
Memoisation	O(n)	O(n)
Iteration	O(n)	O(1)

Here, φ represents the golden ratio.

11. Deployment Decision

I would select the iterative method for calculating the number of ways for 1,000 steps.

Reasons:

1. It avoids recursion depth errors.
2. It requires constant additional memory.
3. It provides linear time complexity.
4. It is suitable for large inputs.

Q7. Day-4 Design Challenge – Diagnose, Redesign and Validate (13 Marks)

For this question, I have selected **Q6 – Recursion and Its Limits** because the naive recursive solution performs repeated calculations.

A. Diagnose the Bottleneck (2 Marks)

The original recursive program calculates the same subproblems multiple times.

For example, ways(3) can be calculated through different recursive paths.

This increases the number of function calls as the input grows.

Evidence:

n	Naive Function Calls
10	177
20	21,891
25	242,785
30	2,692,537

Therefore, repeated calculations are the main performance bottleneck.

B. Define the Design Objective (2 Marks)

The objective is to reduce unnecessary calculations and improve execution time while maintaining the correctness of the result.

The redesigned program should:

- Calculate each required value only once.
- Support large staircase inputs.
- Reduce memory consumption.
- Avoid recursion depth errors.

C. Design the Revised Solution (4 Marks)

I redesigned the original recursive program using an iterative approach.

The iterative approach stores only the last two calculated values.

Algorithm:

1. Initialize two variables with 1.
2. Start a loop from 2 to n.
3. Calculate the next value by adding the previous two values.
4. Update both variables.
5. Return the final result.

Pseudocode:

START

Read n

Initialize a = 1
Initialize b = 1

FOR i = 2 TO n

    c = a + b

    a = b
    b = c

END FOR

Print b

STOP

D. Predict Before Implementing (1 Mark)

I predict that the iterative solution will perform better than naive recursion.

It should reduce the time complexity from exponential to linear and use constant additional memory.

E. Implement and Validate (2 Marks)

Original Program:

```
def ways_naive(n):

    if n <= 1:
        return 1

    return ways_naive(n-1) + ways_naive(n-2)
```

Redesigned Program:

```
def ways_loop(n):

    if n <= 1:
        return 1

    a = 1
    b = 1

    for i in range(2, n+1):

        c = a + b

        a = b
        b = c

    return b
```

print("Number of ways:", ways_loop(1000))

Validation Program:

```
for n in range(1, 7):

    print(
        "Steps:", n,
        "Ways:", ways_loop(n)
    )
```

Output:

Steps: 1 Ways: 1

Steps: 2 Ways: 2

Steps: 3 Ways: 3

Steps: 4 Ways: 5

Steps: 5 Ways: 8

Steps: 6 Ways: 13

```
    b = 1

    for i in range(2, n+1):

        c = a + b

        a = b
        b = c

    return b



for n in range(1, 7):

    print(
        "Steps:", n,
        "Ways:", ways_loop(n)
    )
```

```
Steps: 1 Ways: 1
Steps: 2 Ways: 2
Steps: 3 Ways: 3
Steps: 4 Ways: 5
Steps: 5 Ways: 8
Steps: 6 Ways: 13
```

Comparison:

Parameter	**Original**	**Redesigned**
Method	Recursion	Iteration
Time Complexity	$O(\varphi^n)$	O(n)
Space Complexity	O(n)	O(1)
Repeated calculations	Yes	No
Recursion error	Possible	No
Large input	Inefficient	Efficient

F. Defend the Final Design (2 Marks)

The redesigned iterative solution is more efficient than the original naive recursive solution.

It avoids repeated calculations and recursion depth errors.

It also uses constant additional memory because only two previous values are stored.

The original recursive solution is useful for understanding recursion and mathematical relationships, but the iterative solution is more suitable for large inputs.

Therefore, I selected iteration as the final design because it provides better performance, memory efficiency and reliability.